

```

import asyncio
import logging
from datetime import datetime
import yfinance as yf
import pandas as pd
import numpy as np
from telegram.ext import ApplicationBuilder, CommandHandler, ContextTypes
import ta

# ===== CONFIG =====
BOT_TOKEN = "YOUR_BOT_TOKEN_HERE" # ← توكن البوت من BotFather
CHANNEL_ID = "@your_channel" # ← ID القناة أو chat_id
INTERVAL_MINUTES = 60 # كل كم دقيقة يبعث إشارة

logging.basicConfig(
    format='%(asctime)s - %(levelname)s - %(message)s',
    level=logging.INFO
)
logger = logging.getLogger(__name__)

# ===== جلب البيانات =====
def get_btc_data(interval="15m", period="5d"):
    """جلب بيانات BTC/USD من Yahoo Finance"""
    try:
        ticker = yf.Ticker("BTC-USD")
        df = ticker.history(period=period, interval=interval)
        if df.empty:
            raise ValueError("No data returned")
        return df
    except Exception as e:
        logger.error(f"خطأ في جلب البيانات ({interval}): {e}")
        return None

# ===== المؤشرات =====
def calculate_indicators(df):
    close = df['Close']
    high = df['High']
    low = df['Low']

    # Trend
    df['EMA9'] = ta.trend.EMAIndicator(close, window=9).ema_indicator()
    df['EMA21'] = ta.trend.EMAIndicator(close, window=21).ema_indicator()

```

```

df['EMA50'] = ta.trend.EMAIndicator(close, window=50).ema_indicator()
df['EMA200'] = ta.trend.EMAIndicator(close, window=200).ema_indicator()

# Momentum
df['RSI'] = ta.momentum.RSIIndicator(close, window=14).rsi()
macd = ta.trend.MACD(close, window_fast=12, window_slow=26, window_sign=9)
df['MACD'] = macd.macd()
df['MACD_Signal'] = macd.macd_signal()
df['MACD_Hist'] = macd.macd_diff()

stoch = ta.momentum.StochasticOscillator(high, low, close, window=14, smooth_
df['Stoch_K'] = stoch.stoch()
df['Stoch_D'] = stoch.stoch_signal()

# Volatility
bb = ta.volatility.BollingerBands(close, window=20, window_dev=2)
df['BB_Upper'] = bb.bollinger_hband()
df['BB_Lower'] = bb.bollinger_lband()
df['BB_Mid'] = bb.bollinger_mavg()
df['BB_Width'] = (df['BB_Upper'] - df['BB_Lower']) / df['BB_Mid'] # squeeze
df['ATR'] = ta.volatility.AverageTrueRange(high, low, close, window=14).avera

# Volume
df['OBV'] = ta.volume.OnBalanceVolumeIndicator(close, df['Volume']).on_bal
df['Volume_MA'] = df['Volume'].rolling(20).mean()

# Pivot Points
df['Pivot'] = (high.shift(1) + low.shift(1) + close.shift(1)) / 3
df['R1'] = 2 * df['Pivot'] - low.shift(1)
df['S1'] = 2 * df['Pivot'] - high.shift(1)
df['R2'] = df['Pivot'] + (high.shift(1) - low.shift(1))
df['S2'] = df['Pivot'] - (high.shift(1) - low.shift(1))
df['R3'] = high.shift(1) + 2 * (df['Pivot'] - low.shift(1))
df['S3'] = low.shift(1) - 2 * (high.shift(1) - df['Pivot'])

return df

# ===== تحليل فريم واحد =====
def analyze_timeframe(df, label):
    """تفاصيل + score + direction يحلل فريم واحد ويرجع"""
    df = calculate_indicators(df)
    last = df.iloc[-1]
    price = last['Close']

    score_buy = 0
    score_sell = 0

```

```

details      = []

# --- RSI ---
rsi = last['RSI']
if rsi < 30:
    score_buy += 25; details.append(f"RSI تشبع بيعي ({rsi:.0f}) 🟢")
elif rsi < 45:
    score_buy += 12; details.append(f"RSI منطقة شراء ({rsi:.0f})")
elif rsi > 70:
    score_sell += 25; details.append(f"RSI تشبع شرائي ({rsi:.0f}) 🟡")
elif rsi > 55:
    score_sell += 12; details.append(f"RSI منطقة بيع ({rsi:.0f})")

# --- MACD ---
if last['MACD'] > last['MACD_Signal'] and last['MACD_Hist'] > 0:
    score_buy += 20; details.append("MACD إيجابي 📈")
elif last['MACD'] < last['MACD_Signal'] and last['MACD_Hist'] < 0:
    score_sell += 20; details.append("MACD سلبي 📉")

# --- EMA Stack ---
if last['EMA9'] > last['EMA21'] > last['EMA50']:
    score_buy += 20; details.append("EMAs مرتبة صعوداً 📈")
elif last['EMA9'] < last['EMA21'] < last['EMA50']:
    score_sell += 20; details.append("EMAs مرتبة هبوطاً 📉")

# --- Price vs EMA200 ---
if price > last['EMA200']:
    score_buy += 15; details.append("فوق EMA200 ✅")
else:
    score_sell += 15; details.append("تحت EMA200 ⚠️")

# --- Bollinger Bands ---
if price <= last['BB_Lower']:
    score_buy += 15; details.append("السفلي Band عند 🟢")
elif price >= last['BB_Upper']:
    score_sell += 15; details.append("العلوي Band عند 🟡")

# --- Stochastic ---
if last['Stoch_K'] < 20:
    score_buy += 10; details.append("Stoch بيعي تشبع")
elif last['Stoch_K'] > 80:
    score_sell += 10; details.append("Stoch شرائي تشبع")

# --- Volume Confirmation ---
if last['Volume'] > last['Volume_MA'] * 1.5:
    if score_buy > score_sell:
        score_buy += 10; details.append("حجم تداول مرتفع 🌟")

```

```

        else:
            score_sell += 10; details.append("حجم تداول مرتفع 🌟")

total = score_buy + score_sell
if total == 0:
    return "NEUTRAL", 0, 0, 0, details, last

direction = "BUY" if score_buy > score_sell else "SELL"
confidence = round(max(score_buy, score_sell) / total * 100)

return direction, confidence, score_buy, score_sell, details, last

# ===== Price Action =====
def detect_patterns(df):
    """كشف أنماط الشموع"""
    patterns = []
    c = df.iloc[-1]
    p1 = df.iloc[-2]
    p2 = df.iloc[-3]

    body = abs(c['Close'] - c['Open'])
    candle_size = c['High'] - c['Low']
    upper_wick = c['High'] - max(c['Open'], c['Close'])
    lower_wick = min(c['Open'], c['Close']) - c['Low']

    if candle_size == 0:
        return patterns

    # Bullish Engulfing
    if (p1['Close'] < p1['Open'] and c['Close'] > c['Open'] and
        c['Close'] > p1['Open'] and c['Open'] < p1['Close']):
        patterns.append(("BUY", "Bullish Engulfing 🕯️", 80))

    # Bearish Engulfing
    if (p1['Close'] > p1['Open'] and c['Close'] < c['Open'] and
        c['Close'] < p1['Open'] and c['Open'] > p1['Close']):
        patterns.append(("SELL", "Bearish Engulfing 🕯️", 80))

    # Hammer
    if lower_wick > 2 * body and upper_wick < body * 0.5 and c['Close'] > c['Open']:
        patterns.append(("BUY", "Hammer 🛡️", 72))

    # Shooting Star
    if upper_wick > 2 * body and lower_wick < body * 0.5 and c['Close'] < c['Open']:
        patterns.append(("SELL", "Shooting Star ⭐", 72))

```

```

# Doji
if body < 0.05 * candle_size:
    patterns.append(("NEUTRAL", "Doji ⚖️ - 50 , "تردد"),

# Three White Soldiers
if (all(df.iloc[i]['Close'] > df.iloc[i]['Open'] for i in [-3, -2, -1]) and
    df.iloc[-1]['Close'] > df.iloc[-2]['Close'] > df.iloc[-3]['Close']):
    patterns.append(("BUY", "Three White Soldiers 🇬🇪🇬🇪🇬🇪", 85))

# Three Black Crows
if (all(df.iloc[i]['Close'] < df.iloc[i]['Open'] for i in [-3, -2, -1]) and
    df.iloc[-1]['Close'] < df.iloc[-2]['Close'] < df.iloc[-3]['Close']):
    patterns.append(("SELL", "Three Black Crows 🇬🇪🇬🇪🇬🇪", 85))

# Morning Star
if (p2['Close'] < p2['Open'] and
    abs(p1['Close'] - p1['Open']) < 0.3 * abs(p2['Close'] - p2['Open']) a
    c['Close'] > c['Open'] and c['Close'] > (p2['Open'] + p2['Close']) /
    patterns.append(("BUY", "Morning Star 🌅", 78))

# Evening Star
if (p2['Close'] > p2['Open'] and
    abs(p1['Close'] - p1['Open']) < 0.3 * abs(p2['Close'] - p2['Open']) a
    c['Close'] < c['Open'] and c['Close'] < (p2['Open'] + p2['Close']) /
    patterns.append(("SELL", "Evening Star 🌇", 78))

return patterns

# ===== Multi-Timeframe Analysis =====
def multi_timeframe_analysis():
    """تحليل 3 فريمات وإيجاد التوافق"""

    # جلب البيانات للفريمات الثلاثة
    frames = {
        "Scalping (15m) ⚡": get_btc_data("15m", "3d"),
        "Intraday (1h) 🕒": get_btc_data("1h", "10d"),
        "Swing (1d) 🇬🇪🇬🇪": get_btc_data("1d", "90d"),
    }

    results = {}
    all_buy = 0
    all_sell = 0
    price_latest = None
    atr_latest = None

    for label, df in frames.items():

```

```

if df is None or len(df) < 55:
    logger.warning(f"بيانات غير كافية: {label}")
    continue

direction, conf, sb, ss, details, last = analyze_timeframe(df, label)
results[label] = {
    "direction": direction,
    "confidence": conf,
    "score_buy": sb,
    "score_sell": ss,
    "details": details[:3],
    "price": last['Close'],
    "rsi": last['RSI'],
    "atr": last['ATR'],
    "r1": last['R1'], "r2": last['R2'], "r3": last['R3'],
    "s1": last['S1'], "s2": last['S2'], "s3": last['S3'],
}

if direction == "BUY":
    all_buy += conf
elif direction == "SELL":
    all_sell += conf

# استخدم أحدث سعر (15m)
if label.startswith("Scalping"):
    price_latest = last['Close']
    atr_latest = last['ATR']
    pa_patterns = detect_patterns(df)

if not results:
    return None

# قرار نهائي بناءً على التوافق
buy_count = sum(1 for r in results.values() if r['direction'] == "BUY")
sell_count = sum(1 for r in results.values() if r['direction'] == "SELL")

if buy_count == 3:
    final = "BUY"; confluence = "🔥!توافق كامل على 3 فريمات"; base_conf = 90
elif buy_count == 2:
    final = "BUY"; confluence = "✅!توافق على فريمين"; base_conf = 70
elif sell_count == 3:
    final = "SELL"; confluence = "🔥!توافق كامل على 3 فريمات"; base_conf = 90
elif sell_count == 2:
    final = "SELL"; confluence = "✅!توافق على فريمين"; base_conf = 70
else:
    final = "NEUTRAL"; confluence = "🕒!تعارض بين الفريمات"; base_conf = 0

```

```

return {
    "final": final,
    "confluence": confluence,
    "base_conf": base_conf,
    "results": results,
    "price": price_latest,
    "atr": atr_latest,
    "pa_patterns": pa_patterns if 'pa_patterns' in locals() else [],
    "buy_count": buy_count,
    "sell_count": sell_count,
}

```

===== حساب الأهداف =====

```

def calculate_targets(analysis):
    price      = analysis['price']
    atr        = analysis['atr']
    direction  = analysis['final']
    r          = analysis['results']

```

إذا موجود (1d) Swing من الـ Pivot Points استخدم

```

swing_key = next((k for k in r if "Swing" in k), None)
res = r[swing_key] if swing_key else list(r.values())[0]

```

if direction == "BUY":

```

    sl = round(price - 1.5 * atr, 0)
    tp1 = round(price + 1.0 * atr, 0)
    tp2 = round(price + 2.2 * atr, 0)
    tp3 = round(price + 4.0 * atr, 0)

```

إذا قريبة (±0.5%) Pivot R levels اضبط على

```

for level, attr in [(res['r1'], 'tp1'), (res['r2'], 'tp2'), (res['r3'], '
    if level and abs(level - locals()[attr]) / price < 0.005:
        locals()[attr] = round(level, 0)

```

else:

```

    sl = round(price + 1.5 * atr, 0)
    tp1 = round(price - 1.0 * atr, 0)
    tp2 = round(price - 2.2 * atr, 0)
    tp3 = round(price - 4.0 * atr, 0)

```

risk = abs(price - sl)

reward = abs(tp2 - price)

rr = round(reward / risk, 2) if risk > 0 else 0

```

return {
    'entry': round(price, 0),

```

```

        'sl': sl, 'tp1': tp1, 'tp2': tp2, 'tp3': tp3,
        'rr': rr, 'atr': round(atr, 0)
    }

# ===== بناء الرسالة =====
def build_message(analysis):
    if not analysis or analysis['final'] == "NEUTRAL":
        return None

    t = calculate_targets(analysis)
    now = datetime.now().strftime("%Y-%m-%d %H:%M")

    if analysis['final'] == "BUY":
        emoji = "🟢"; dir_ar = "BUY شراء 📈"
    else:
        emoji = "🔴"; dir_ar = "SELL بيع 📉"

    conf = analysis['base_conf']
    conf_bar = "▀" * (conf // 10) + "░" * (10 - conf // 10)
    price_fmt = f"${t['entry']:,}.0f"

    msg = f"""\n{emoji}{emoji} إشارة بيتكوين | BTC/USD {emoji}{emoji}

=====
📊 الاتجاه: {dir_ar}
💰 سعر الدخول: {price_fmt}
🕒 التوقيت: {now}

===== 🎯 الأهداف =====
✅ TP1 (هدف أول): ${t['tp1']:,}.0f
✅ TP2 (هدف ثاني): ${t['tp2']:,}.0f
✅ TP3 (هدف ثالث): ${t['tp3']:,}.0f
🛑 SL (وقف خسارة): ${t['sl']:,}.0f
⚖️ Risk/Reward: 1:{t['rr']}

===== 📊 تحليل الفريمات =====

for label, res in analysis['results'].items():
    d = res['direction']
    icon = "🟢" if d == "BUY" else "🔴" if d == "SELL" else "⚪"
    msg += f"\n{icon} {label}: {d} ({res['confidence']}%)"
    for det in res['details'][:2]:
        msg += f"\n    ↳ {det}"

msg += f"""\n

===== 🕯️ Price Action =====

```



```

pa_shown = [p for p in analysis['pa_patterns'] if p[0] != "NEUTRAL"][:2]
if pa_shown:
    for p in pa_shown:
        msg += f"\n• {p[1]}"
else:
    msg += "\nلا يوجد نمط واضح"

msg += f""

```

————— 💡 قوة الإشارة —————

```

{conf_bar} {conf}%
{analysis['confluence']}
ATR: ${t['atr']:.0f}

```

⚠️ للأغراض التعليمية فقط
تداول بمسؤولية وادرس المخاطر 📖

```

return msg.strip()

```

===== كومانداات =====

```

async def start(update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("مرحباً في بوت إشارات البيتكوين 🐼")

```

📊 يحلل 3 فريمات زمنية معاً:

- ⚡ Scalping - 15 دقيقة
- 🕒 Intraday - ساعة
- 🇸🇦 17 Swing - يومي

📌 الأوامر:

```

/analyze - تحليل فوري الآن
/status - الحالي BTC سعر
/help - مساعدة

```

🔔 إشارات تلقائية كل ساعة

```

async def analyze_now(update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("...جاري تحليل البيتكوين على 3 فريمات 🕒")
    try:
        analysis = multi_timeframe_analysis()
        if not analysis:
            await update.message.reply_text("❌ فشل جلب البيانات، حاول مرة ثانية")
            return

        msg = build_message(analysis)

```

```

if msg:
    await update.message.reply_text(msg)
else:
    price = f"${analysis['price']:, .0f}" if analysis['price'] else "N/A"
    await update.message.reply_text(
        f"🕒 لا توجد إشارة واضحة الآن\n"
        f"💰 سعر BTC: {price}\n"
        f"📈 {analysis['results'].get('Swing (1d) 📅', {}).get('direction', '')}\n"
        f"🕒 {analysis['results'].get('Intraday (1h) 🕒', {}).get('direction', '')}\n"
        f"⚡ {analysis['results'].get('Scalping (15m) ⚡', {}).get('direction', '')}\n"
        f"⌚ الفرييمات متعارضة - انتظر توافق أوضح"
    )
except Exception as e:
    logger.error(f"خطأ: {e}")
    await update.message.reply_text(f"❌ خطأ: {str(e)}")

async def status(update, context: ContextTypes.DEFAULT_TYPE):
    try:
        df = get_btc_data("1h", "2d")
        if df is not None:
            price = df['Close'].iloc[-1]
            open_ = df['Close'].iloc[0]
            change = price - open_
            pct = (change / open_) * 100
            arrow = "📈" if change > 0 else "📉"
            await update.message.reply_text(
                f"💰 BTC/USD الآن\n\n"
                f"💰 السعر: ${price:, .0f}\n"
                f"{arrow} التغير (24h): ${change:+, .0f} ({pct:+.2f}%) \n"
                f"🕒 آخر تحديث: {datetime.now().strftime('%H:%M')} \n"
                f"⌚ إشارات تلقائية كل {INTERVAL_MINUTES} دقيقة"
            )
        else:
            await update.message.reply_text("⚠️ فشل جلب السعر")
    except Exception as e:
        await update.message.reply_text(f"❌ خطأ: {e}")

# ===== إشارات تلقائية =====
async def auto_signals(context):
    try:
        analysis = multi_timeframe_analysis()
        if not analysis:
            return

        msg = build_message(analysis)

```

```

    if msg:
        await context.bot.send_message(chat_id=CHANNEL_ID, text=msg)
        logger.info("✅ أُرسلت إشارة BTC")
    else:
        logger.info(f"🚫 لا توافق - BUY:{analysis['buy_count']} SELL:{analysis['sell_count']}")
except Exception as e:
    logger.error(f"❌ خطأ في الإشارة التلقائية: {e}")

# ===== تشغيل =====

def main():
    app = ApplicationBuilder().token(BOT_TOKEN).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("analyze", analyze_now))
    app.add_handler(CommandHandler("status", status))

    app.job_queue.run_repeating(
        auto_signals,
        interval=INTERVAL_MINUTES * 60,
        first=15
    )

    logger.info(f"🔄 BTC Bot يعمل - إشارات كل {INTERVAL_MINUTES} دقيقة")
    app.run_polling()

if __name__ == "__main__":
    main()

```